

## BACKEND MODULE DESIGN DOCUMENT

### PART-2: ENTITY, REPOSITORY, SERVICE & CONTROLLER STANDARDS

# School ERP

Java 21 + Spring Boot 3 | Spring Data JPA + Hibernate | PostgreSQL

Prepared by: Backend Architecture Team

Document Version: 1.0

Date: 29 July 2026

*Classification: Confidential — Engineering Team Only*

## Preamble

---

This document is Part-2 of the Backend Module Design Document (BMDD) v1.0 for the School ERP System, continuing directly from Part-1: Backend Module Architecture. It defines Java/Spring-Boot-specific implementation standards for the Entity, Repository, Service, and Controller layers already introduced in BMDD Part-1 Sections 4-5, consolidating MDB Part-3 Sections 22-25, DDD Parts 1-3, and ASD Parts 1-3 into layer-specific implementation guidance.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, any MDB Part, any DDD Part, any ASD Part, or BMDD Part-1. It contains no source code, no Java classes, no Entity/Repository/Service/Controller implementation, no UML, no sequence diagrams, and no code examples — every standard is stated as a rule only.

## 11. Entity Design Standards

---

Fixes JPA entity implementation conventions within the entity sub-package (BMDD Part-1 Section 4), extending MDB Part-3 Section 22 and DDD Part-1 Sections 7-9.

- One entity class per logical entity already defined in Appendix-H, named identically to that entity's Appendix-G/H name — an entity class is never split across multiple classes, and no entity class represents more than one logical entity.
- The Common Columns baseline (surrogate ID, version, audit columns, soft-delete — DDD Part-1 Section 7) is implemented via a shared base entity class or equivalent, inherited by every entity, rather than repeated field-by-field in each entity class.
- Relationship annotations map exactly to the cardinality and mandatory/optional status already fixed per relationship in Appendix-H Section 5 and DDD Part-2 Section 13 — an entity class's relationship mapping is never adjusted for developer convenience in a way that changes the effective cardinality.
- An entity class contains no business logic — computed, non-persisted values are computed in the Service layer (Section 13) or Mapper (BMDD Part-1 Section 8), never as logic embedded in the entity class itself.
- Polymorphic association fields (owner\_type/owner\_ref\_id — DDD Part-1 Section 9) are typed and documented consistently across every entity using the pattern, with the application-layer integrity check (Section 18 this document) implemented once, in a shared location, rather than duplicated per entity.

## 12. Repository Standards

---

Fixes Repository interface implementation conventions, extending MDB Part-3 Section 23 and DDD Part-3.

- One Spring Data JPA repository interface per entity, named <EntityName>Repository, residing in the module's repository sub-package (BMDD Part-1 Section 4) — matching the one-repository-per-entity rule already fixed in MDB Part-3 Section 23.
- Derived query methods (method-name-based queries) are preferred over explicit query annotations wherever the method name remains clear and unambiguous; an explicit, structured query is used only where a derived method name would become unreasonably long or cannot express the needed condition.
- Every method returning a potentially high-volume result (DDD Part-2 Section 11 Transaction Data classification) accepts a pagination parameter and returns a paginated result type, per MDB Part-3 Section 23 and ASD Part-3 Section 23 — a repository method is never added that returns an unbounded collection for a high-volume entity.
- A repository interface contains no default-method business logic — a repository is purely a data-access contract; any logic beyond a direct data-access query belongs in the Service layer (Section 13).

## 13. Service Standards

---

Fixes Service component implementation conventions, extending BMDD Part-1 Section 5 and MDB Part-3 Section 24.

- A module's Service layer is split into an interface and an implementation class wherever that Service is published for another module's dependency (BMDD Part-1 Section 6) — a Service consumed only within its own module may be a single concrete class without a separate interface, but a cross-module-published Service is never exposed as a concrete class directly.
- Service methods are granular and named for the specific business operation they perform (e.g., a distinct method per Business Rule-bearing operation) rather than a small number of broad, multi-purpose methods — this granularity directly supports the Unit Testing requirement already fixed in Appendix-M Section 9 and MDB Part-3 Section 29.
- Every Service method that changes state follows the fixed sequence already established in MDB Part-3 Section 24: validate (Section 15 this document) → enforce Business Rules (Section 18) → persist via Repository → call AuditLogService → raise domain events where applicable (BMDD Part-1 Section 9) — this sequence is not reordered per module.
- A Service method depends only on Repository interfaces (its own module's) and Service interfaces (other modules', per BMDD Part-1 Section 6) — constructor injection only, per MDB Part-1 Section 10.1, with no field injection anywhere in the Service layer.

## 14. Controller Standards

---

Fixes Controller component implementation conventions, extending BMDD Part-1 Section 5 and ASD Parts 1-3.

- One Controller class per module's set of related API Groups (Appendix-K Section 18), organized so that each Controller method corresponds to exactly one API operation already specified in ASD Parts 1-3 — a Controller method never implements more than one logical operation.
- A Controller method's body is limited to: receiving the validated request DTO (Section 15), invoking exactly one Service method, and returning the Service result wrapped in the standard response structure (ASD Part-1 Section 10) — no conditional business logic, no loop over Business Rule checks, and no direct Repository or Mapper invocation.
- Authorization annotations (ASD Part-2 Section 12) are declared at the Controller method level, matching the Role Permission already fixed in Appendix-E Field 38 for that specific operation — a Controller method is never left without an explicit authorization declaration, even where the intended access is broad.
- A Controller class depends only on its own module's Service layer (Section 13) — never on another module's Service, Repository, or Entity, preserving the one-Controller-to-one-module rule already fixed in BMDD Part-1 Section 6.

## 15. Validation Flow Standards

---

Fixes the concrete implementation sequence realizing the Validation Standards already established in MDB Part-2 Section 16 and ASD Part-3 Section 21.

- Bean Validation constraints on the Request DTO (BMDD Part-1 Section 7) are triggered automatically at the Controller boundary, before the Controller method body executes — a Controller never manually re-checks a field-level rule already expressed as a Bean Validation constraint.
- A Bean Validation failure short-circuits the request before it reaches the Service layer, returning the Validation Error response category (ASD Part-1 Section 10) with every failing field identified together (ASD Part-3 Section 21) — the Service method is never invoked with a request known to have failed field-level validation.
- Cross-field validation not expressible as a simple Bean Validation constraint (e.g., a date-range check) is implemented as an explicit, named validation step at the start of the relevant Service method, distinct from Business Rule enforcement (Section 18), which follows only after this step passes.

## 16. Exception Handling Standards

Fixes the concrete implementation of the five exception categories already established in MDB Part-2 Section 17 and ASD Part-3 Section 28.

| Category                   | Raised By                                                                                | Handled By                                                                                                         |
|----------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Validation Exception       | Bean Validation framework, automatically (Section 15)                                    | Centralized global exception handler — never a per-Controller try/catch                                            |
| Business Rule Exception    | Service layer, explicitly, referencing the specific Appendix-C v1.1 rule ID (Section 18) | Same centralized handler                                                                                           |
| Integration Exception      | Event Publisher/ChannelAdapter boundary (BMDD Part-1 Section 9)                          | Caught at that boundary; never propagates to the Controller uncaught                                               |
| Concurrency Exception      | JPA optimistic-locking version mismatch (DDD Part-1 Section 4)                           | Same centralized handler                                                                                           |
| Unhandled/System Exception | Any uncaught runtime failure                                                             | Catch-all in the same centralized handler; full detail logged server-side only (Section 20, MDB Part-2 Section 20) |

A single, centralized exception-handling component serves every module's Controllers — a module never implements its own, separate exception-handling logic, preserving the uniform Error Response contract already fixed in ASD Part-1 Section 10.

## 17. Transaction Management Standards

---

Fixes the concrete Service-method-level implementation of Transaction Management already established in MDB Part-3 Section 25.

- A state-changing Service method is the transaction boundary — declarative transaction management wraps the entire method, including every Repository call and AuditLogService write it makes, so the operation commits or rolls back as a single unit.
- When a Service method calls another module's published Service method (BMDD Part-1 Section 6/9) within the same logical operation, the called method participates in the caller's existing transaction rather than starting an independent one, unless the specific cross-module effect is deliberately event-based (BMDD Part-1 Section 9), in which case it is intentionally outside the caller's transaction.
- Read-only Service methods (Reports module, Search operations — ASD Part-3 Section 26) are explicitly marked read-only at the transaction-annotation level, consistent with MDB Part-3 Section 25's read-replica routing optimization.
- The pessimistic-locking cases already identified for safety-critical operations (BR-ADM-001/BR-ADM-007 seat allocation — MDB Part-3 Section 25, DDD Part-4 Section 36) are implemented at the specific Repository call within the transaction, not as a broader table-level or module-level lock.

## 18. Business Rule Enforcement Standards

---

Fixes the definitive, implementation-level discipline for where and how all 71 Business Rules (Appendix-C v1.1) are enforced, consolidating MDB Part-3 Section 24, DDD Part-3 Section 21, and ASD Part-3 Section 22.

- Every Business Rule is enforced in exactly one Service method, identified by its Appendix-C v1.1 rule ID in that method's documentation comment (MDB Part-3 Section 29) — a rule is never partially enforced in one Service method and completed in another, and never duplicated across two Service methods.
- A Business Rule is never enforced as a database constraint (DDD Part-3 Section 21/23) and never left to Controller-level or frontend-only validation (Section 15) — the Service layer is its sole enforcement point, without exception.
- Critical-priority Business Rules (19 of 71, Appendix-C v1.1 Section 2.2) receive no different implementation pattern from other rules — priority affects testing rigor (Appendix-M Section 23) and code-review attention, not where or how the rule is technically enforced.
- A Business Rule requiring cross-module data (e.g., BR-FEE-005 checking a waiver ceiling against Fee Structure) is enforced in the Service method of the module that owns the rule's outcome, calling the other module's published Service interface (Section 6, BMDD Part-1) to obtain the data it needs — the rule's enforcement location is never split across two modules' Service layers.

## 19. Layer Interaction Standards

---

Fixes the fixed, non-diagram description of how a single request flows through the layers already defined in BMDD Part-1 Section 5, consolidating this document's Sections 11-18 into one end-to-end statement.

- A request arrives at the Controller (Section 14), which delegates immediately to a single Service method after Bean Validation (Section 15) has already passed automatically.
- The Service method enforces any applicable Business Rules (Section 18), then calls its own module's Repository (Section 12) — and, where a cross-module dependency exists, another module's published Service interface (BMDD Part-1 Section 6) — all within the transaction boundary already fixed in Section 17.
- Before returning, the Service method calls AuditLogService and, where applicable, raises a domain event (BMDD Part-1 Section 9); it then hands its result to the Mapper (BMDD Part-1 Section 8) to produce the Response DTO.
- The Controller receives the Response DTO unchanged and returns it wrapped in the standard response structure (ASD Part-1 Section 10) — no layer downstream of the Controller is ever skipped, and no layer performs another layer's responsibility (Section 5, BMDD Part-1) even under time pressure or for a seemingly trivial operation.



## 20. Backend Quality Checklist

Verified for every new or changed Entity, Repository, Service, or Controller component, complementing the Backend Review Checklist already fixed in BMDD Part-1 Section 10.

| # | Checklist Item                                                                                                                       |
|---|--------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Entity class (Section 11) maps one-to-one to its Appendix-H logical entity, with the Common Columns baseline inherited, not repeated |
| 2 | Repository (Section 12) returns paginated results for any high-volume entity and contains no business logic                          |
| 3 | Service method (Section 13) follows the fixed validate→enforce→persist→audit→event sequence with no reordering                       |
| 4 | Controller method (Section 14) maps to exactly one API operation with an explicit authorization declaration                          |
| 5 | Validation flow (Section 15) short-circuits correctly before Business Rule evaluation                                                |
| 6 | Exceptions (Section 16) are raised from the correct category and handled only by the centralized handler                             |
| 7 | Transaction boundary (Section 17) covers the full state-changing operation with no partial-commit risk                               |
| 8 | Every touched Business Rule (Section 18) is enforced in exactly one Service method, referenced by its Appendix-C v1.1 ID             |
| 9 | No layer in the request flow (Section 19) is skipped or performs another layer's responsibility                                      |